CS1200: Intro. to Algorithms and their Limitations	Prof. Anurag Anshu

Lecture 17: Resolution

Harvard SEAS - Spring 2026 *2026-04-06*

1 Announcements

- Prof. Adam Hesterberg teaching today!
- SRE5 on Wed.
- PS7 due tomorrow
- Hesterberg-Vadhan Chapter 16.3, 17

2 Modelling using Satisfiability

One reason that SAT is important is its richness in encoding other problems. Thus any improvements to algorithms for (CNF-)SAT (aka "SAT Solvers") can be easily be applied to many other problems we want to solve.

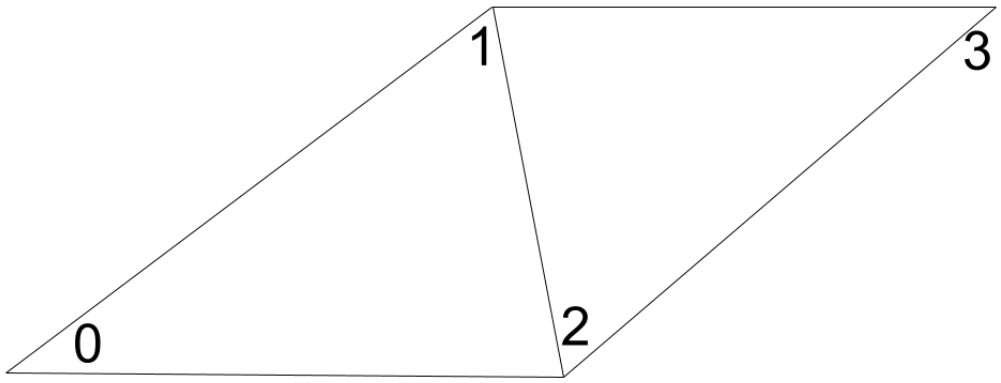
Theorem 2.1. Graph Coloring *with k colors on graphs with n nodes and m edges can be reduced in time $O(n + km)$ to* CNF-Satisfiability *with kn variables and $n + km$ clauses.*

Proof. Given $G = (V, E)$ and $k \in \mathbb{N}$, we will construct a CNF formula ϕ_G that captures the constraints of the Graph Coloring problem. The variables of ϕ_G are *indicator variables* $x_{v,i}$ for each $v \in V$ and $i \in [k]$, which intuitively are meant to correspond to vertex v being assigned to color i.

We then have a few types of clauses:

1.
2.
3.

For instance, if G is the graph below and $k = 3$,



then we make the following SAT instance ϕ_G:

We then call the SAT oracle on ϕ_G and get an assignment α. If $\alpha = \perp$, we say G is not k-colorable. Otherwise, we construct and output the coloring f_α given by:

$$f_\alpha(v) =$$

The runtime essentially follows from our description.

For correctness, we make two claims:

Claim 2.2. *If G has a valid k-coloring, then ϕ_G is satisfiable.*

Claim 2.3. *If α satisfies ϕ_G, then f_α is a proper k-coloring of G.*

Both of these claims are worth checking. Note that f_α is *well-defined* because α satisfies clauses of type 1 (so the definition of f_α doesn't try to take the minimum of an empty set) and is *proper* due to clauses of type 2. □

Unfortunately, the fastest known algorithms for SAT have worst-case runtime exponential in n. However, enormous effort has gone into designing heuristics that complete much more quickly on many real-world instances. In particular, SAT Solvers—with many additional optimizations—were used to solve large-scale Graph Coloring instances arising in the 2016 US Federal Communications Commission (FCC) auction to reallocate wireless spectrum.

As we will see repeatedly in the rest of the course, SAT can encode a very wide variety of problems of interest, with the next example being LONGEST PATH in the Sender–Receiver Exercise next week.

Thus motivated, in the next classwe will turn to algorithms for SAT, to get a taste of some of the ideas that go into SAT Solvers.

3 The Basic Resolution Algorithm

SAT Solvers are algorithms to solve CNF-SATISFIABILITY. Although they have worst-case exponential running time, on many "real-world" instances, they terminate surprisingly quickly with either (a) a satisfying assignment, or (b) a "proof" that the input formula is unsatisfiable.

The best known SAT solvers implicitly use the technique of *resolution*. The idea of resolution is to repeatedly derive new clauses from the original clauses (using a valid deduction rule) until we either derive an empty clause (which is false, and thus gives a proof that the original formula is unsatisfiable) or we cannot derive any more clauses (in which case we can efficiently construct a satisfying assignment).

Our deduction rule will make use of a clause simplification procedure `Simplify`, which takes a clause C and simplifies it as follows:

1.

2.

3.

Definition 3.1 (resolution rule)**.** For clauses C and D, define their *resolvent* to be

$$C \diamond D = \begin{cases} \underline{\qquad\qquad\qquad\qquad\qquad\qquad} & \text{if } \ell \text{ is a literal s.t. } \ell \in C \text{ and } \neg\ell \in D \\ 1 & \text{if there is no such literal } \ell \end{cases}$$

Here $C - \{\ell\}$ means remove literal ℓ from clause C, and 1 represents `true`. As we will see below, if C and D can be resolved with respect to more than one literal ℓ, then for all choices of ℓ we will have $\texttt{Simplify}((C - \{\ell\}) \vee (D - \{\neg\ell\})) = 1$, so $C \diamond D$ is well-defined.

In the special case where $C = \ell, D = \neg\ell$, we obtain the empty clause ( ), which is always false:

$$(\ell) \diamond (\neg\ell) = (\ ) = \text{FALSE}.$$

From now on, it will be useful to view a CNF formula as just a set $\mathcal{C}$ of clauses.

Definition 3.2. Let $\mathcal{C}$ be a set of clauses over variables $x_0, \ldots, x_{n-1}$. We say that an assignment $\alpha \in \{0,1\}^n$ *satisfies* $\mathcal{C}$ if α satisfies all of the clauses in $\mathcal{C}$, or equivalently α satisfies the CNF formula

$$\varphi(x_0, \ldots, x_{n-1}) = \bigwedge_{C \in \mathcal{C}} C(x_0, \ldots, x_{n-1}).$$

Intuition. The intuition behind resolution can be seen from the following example. Consider a CNF-SATISFIABILITY instance that includes two clauses $C = (\neg x_0 \vee x_1)$ and $D = (\neg x_1 \vee x_2)$, along with other clauses. Since a satisfying assignment must make both C_1 and C_2 true, there is an implicit dependence between x_0 and x_2:

Following the definition, this is precisely the resolvent:

$$C \diamond D = (\neg x_0 \vee x_1) \diamond (\neg x_1 \vee x_2) =$$

Example 3.3.

$$(x_0 \vee \neg x_1 \vee x_3 \vee \neg x_5) \diamond (x_1 \vee \neg x_4 \vee \neg x_5) =$$

Example 3.4. We could also have a clause that appears to be resolvable in two ways:

$$(x_0 \vee x_1 \vee \neg x_4) \diamond (\neg x_0 \vee x_2 \vee x_4) =$$

The following theorem gives us a criterion to decide if a set of clauses is satisfiable. Note that resolution plays a crucial rule here.

Theorem 3.5 (Resolution Theorem)**.** *Let $\mathcal{C}$ be a set of clauses over n variables $x_0, \ldots, x_{n-1}$. Suppose that $\mathcal{C}$ is* closed under resolution*, meaning that for every $C, D \in \mathcal{C}$, we have $C \diamond D \in \mathcal{C} \cup \{1\}$. Then:*

1. *$(\ ) \in \mathcal{C}$ iff*
2. *If $(\ ) \notin \mathcal{C}$, then* `ExtractAssignment(`$\mathcal{C}$`)` *finds where* `ExtractAssignment` *is an algorithm described in Section 4.*

Thus we can obtain an algorithm for solving CNF-SATISFIABILITY as follows. We start with the set of clauses $C_0, C_1, \cdots, C_{m-1}$ that appear in the CNF φ, simplify all the clauses in φ and then:

1. Resolve C_0 with each of $C_1, \ldots, C_{m-1}$, adding any new clauses obtained from the resolution $C_m, C_{m+1}, \ldots$. If the empty clause $(\ )$ is found, return `unsatisfiable`.
2. Resolve C_1 with each of $C_2, \ldots, C_{m-1}$ as well as with
3. Resolve C_2 with each of $C_3, \ldots, C_{m-1}$ as well as with
4. etc.
5. Run `ExtractAssignment` on the set of all clauses and return the satisfying assignment.

Pseudocode is given as Algorithm 3.2.

```
ResolutionInOrder(φ):
Input  : A CNF formula φ(x_0, ..., x_{n-1})
Output : Whether φ is satisfiable or unsatisfiable
Let C_0, C_1, ..., C_{m-1} be the clauses in φ, after simplifying each clause;
i = 0 ;  /* clause to resolve with others in current iteration */
f = m ;     /* start of 'frontier' - new resolvents from current
  iteration */
g = m ;                                          /* end of frontier */
while f > i + 1 do
    foreach j = i + 1 to f − 1 do
        R = C_i ◇ C_j;
        if R = 0 then return unsatisfiable;
        else if R ∉ {1, C_0, C_1, ..., C_{g-1}} then
            C_g = R;
            g = g + 1;
    f = g;
    i = i + 1
return ExtractAssignment((C_0, C_1, ... C_{g-1}))
```

Algorithm 3.2: A Resolution-based SAT Algorithm

Example 3.6. Let's apply Algorithm 3.2 to the following formula:

$$\phi(x_0, x_1, x_2) = (\neg x_0 \vee x_1) \wedge (\neg x_1 \vee x_2) \wedge (x_0 \vee x_1 \vee x_2) \wedge (\neg x_2)$$

Example 3.7. $\psi(x_0, x_1, x_2, x_3) = (\neg x_0 \vee x_3) \wedge (x_0 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_1) \wedge (\neg x_3)$

.

4 Assignment Extraction

We begin by describing the `ExtractAssignment` algorithm for finding a satisfying assignment to a set $\mathcal{C}$ of clauses that are closed under resolution and don't contain $(\ )$. Specifically, we generate our satisfying assignment one variable v at a time in the following manner:

1. If $\mathcal{C}$ contains a singleton clause (v), then
2. If it contains a singleton clause $(\neg v)$ then
3. If it contains neither (v) nor $(\neg v)$, then
4. $\mathcal{C}$ cannot contain both (v) and $(\neg v)$, because

Once we have assigned a variable to a value, we set that variable's value in every clause and simplify.

Formally, the algorithm works as follows:

```
ExtractAssignment(𝒞):
Input  : A closed and simplified set 𝒞 of clauses over variables
         x_0, ..., x_{n-1} such that ( ) ∉ 𝒞
Output : An assignment α ∈ {0,1}^n that satisfies all of the clauses
         in 𝒞
foreach i = 0, ..., n-1 do
    if (x_i) ∈ 𝒞 then α_i = 1;
    else α_i = 0;
    𝒞 = 𝒞|_{x_i=α_i}, meaning that we set x_i = α_i and then simplify all clauses.
return α
```

Algorithm 4.2: Assignment extraction algorithm

Example 4.1. Consider applying Algorithm 4.2 to the set of clauses derived from the formula in Example 3.7 above:

$$(\neg x_0 \vee x_3), (x_0 \vee \neg x_3), (\neg x_1 \vee x_2), (\neg x_2 \vee x_1), (\neg x_3), (\neg x_0)$$

5 Proof Sketches of Runtime and Correctness

Proof Sketch of Theorem 3.5. First, suppose $\mathcal{C}$ is such that $(\ ) \in \mathcal{C}$.

Conversely, if $(\ ) \notin \mathcal{C}$, then Item 2 says that `ExtractAssignment` will find a satisfying assignment, so $\mathcal{C}$ is satisfiable. Thus we turn to sketching the proof of Item 2. The key point is to show that after assigning a variable x_i as in `ExtractAssignment`, the set of clauses $\mathcal{C}|_{x_i=\alpha_i}$ satisfies the following:

$\square$

Now, we consider the runtime and correctness of the resolution algorithm. Let $\mathcal{C}_{fin}$ be the final set of clauses produced in Algorithm 3.2. Let k_{fin} be the maximum *width* (number of literals) among the clauses in $\mathcal{C}_{fin}$.

Runtime. Before analysing the runtime of the resolution algorithm, let's understand why the resolution algorithm terminates.

We can now give a finer estimate of the runtime of the resolution algorithm.

Correctness:

6 Efficient algorithm for 2SAT

Although the worst-case running time of resolution is exponential, the following is an important special case where it has polynomial runtime.

Input: A CNF formula φ on n variables in which each clause has width at most 2 (i.e. contains at most 2 literals) **Output:** An $\alpha \in \{0,1\}^n$ such that $\varphi(\alpha) = 1$, or $\perp$ if no satisfying assignment exists

Computational Problem 2SAT

Runtime of the resolution algorithm for 2SAT**:** observe that on a 2SAT instance φ, Algorithm 3.2 will never create a clause of size larger than 2:

Corollary 6.1. 2SAT *can be solved in time* _____.

There is an algorithm known for 2SAT with runtime $O(n + m)$, where m is the number of clauses, by reduction to finding strongly connected components of directed graphs. (This algorithm is covered in CS1240.) Unfortunately, just like with coloring, once we switch from $k = 2$ to $k = 3$, the best known algorithms have exponential ($O(c^n)$) worst-case runtimes.

7 SAT Solvers

Enormous effort has gone into designing SAT Solvers that perform well on many real-world satisfiability instances, often but not always avoiding the worst-case exponential complexity. These methods are very related to Resolution. In some sense, they can be viewed as interleaving the `ExtractAssignment` algorithm and Resolution steps, in the hope of quickly finding either a satisfying assignment or a proof of unsatisfiability.